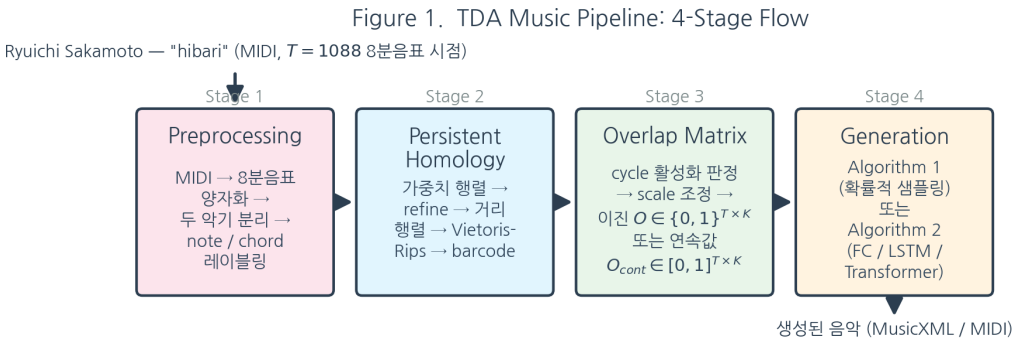
위상수학적 음악 분석 — Step 4

시각자료 (Figures)

본 장은 논문 본문에 삽입될 5개의 정적 figure와 보충자료(supplementary material)로 제공되는 1개의 interactive figure를 모아서 제시한다. 각 figure는 `docs/figures/` 폴더 아래의 별도 스크립트로 재현 가능하다.

#	파일	생성 스크립트	역할	배치
1	`fig1_pipeline.png`	`make_fig1_pipeline.py`	4-stage 파이프라인 흐름도	본문
2	`fig2_cycle3d_interactive.html`	`make_fig2_interactive.py`	발견된 cycle의 3D interactive 시각화	**보충자료**
3	`fig3_tonnetz_hibari.png`	`make_fig3_tonnetz_hibari.py`	Tonnetz 격자에 hibari 음 배치	본문
4	`fig4_barcode.png`	`make_fig4_barcode.py`	Persistence barcode 다이어그램	본문
5	`fig5_pianoroll.png`	`make_fig5_pianoroll.py`	원곡 vs 생성곡 piano roll 비교 (3 구간)	본문
6	`fig6_js_curve.png`	`make_fig6_js_curve.py`	학습 epoch별 JS divergence 곡선	본문

Figure 1 — TDA Music Pipeline: 4-Stage Flow



캡션. 본 연구의 파이프라인은 네 개의 순차적 stage로 구성된다. Stage 1은 MIDI 원본을 8분음표 단위로 양자화하고 두 악기를 분리한 뒤 note / chord 레이블링을 수행한다. Stage 2는 가중치 행렬을 구축하고 refine 과정을 거쳐 거리 행렬을 만들어 Vietoris-Rips 복합체로부터 persistent homology를 계산한다. Stage 3은 발견된 cycle들의 활성화 정보를 이진 중첩행렬 $O \in \{0,1\}^{T\times K}$로 또는 연속값 버전 $O_{\text{cont}} \in [0,1]^{T\times K}$ (2.5절)로 변환한다. Stage 4는 이 중첩행렬을 seed로 하여 Algorithm 1 (확률적 샘플링) 또는 Algorithm 2 (FC / LSTM / Transformer)로 음악을 생성한다.

Figure 2 (Interactive, Supplementary) — hibari Cycles in 3D

본문에서 제외, 별도 HTML 파일 `fig2_cycle3d_interactive.html`.

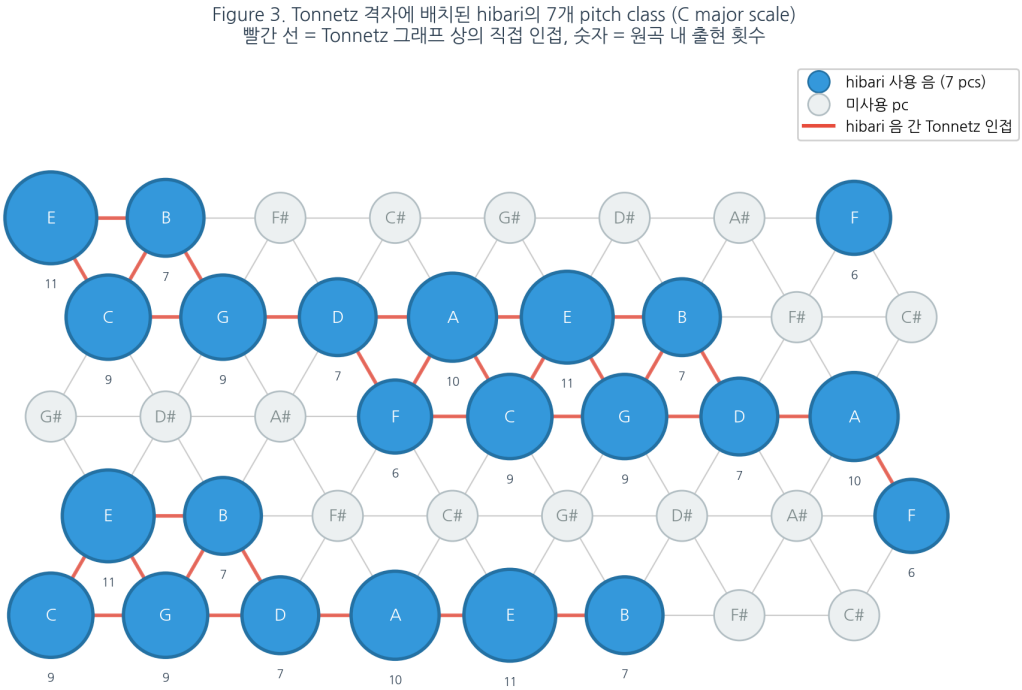
정적 PNG로는 위에서 내려다보는 각도에서만 cycle 간 note 공유 관계가 잘 보이고, 다른 각도에서는 겹침이 심해 해석이 어렵다. 이를 해결하기 위해 Plotly 기반 interactive 3D 시각화를 supplementary로 분리하였다.

사용법.

- 브라우저에서 `docs/figures/fig2_cycle3d_interactive.html` 파일을 연다.
- 마우스 드래그로 자유 회전, 스크롤로 줌.
- 우측 legend를 클릭하면 특정 cycle을 on/off할 수 있다.
- 각 cycle의 선에 마우스를 올리면 해당 cycle에 포함된 note label들이 pitch name + 옥타브 + duration과 함께 hover로 표시된다.
- 기본 카메라는 "위에서 내려다보는" 각도로 설정되어 있어 여러 cycle이 어떤 note들을 공유하는지 한눈에 파악할 수 있다.

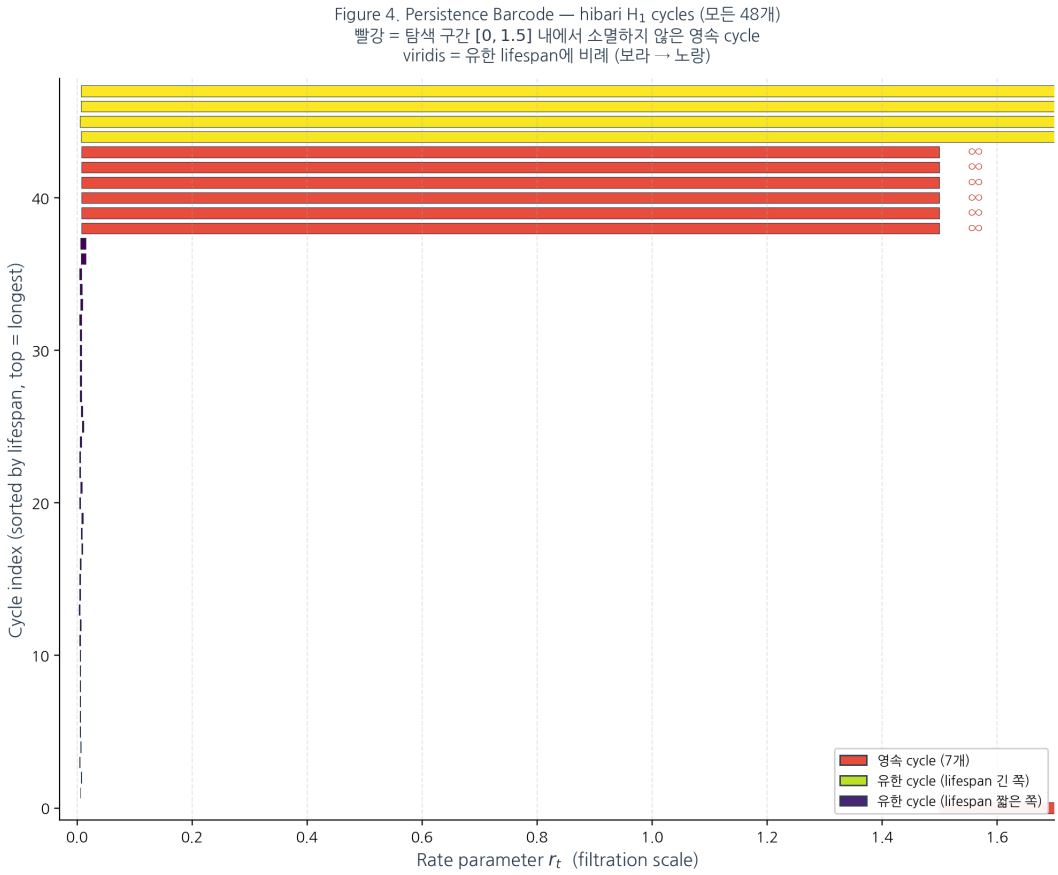
상위 20개 cycle을 색상별로 구분하여 표시한다. 논문 본문에는 이 figure가 들어가지 않으며, PDF 원고와 별개로 supplementary zip에 포함된다.

Figure 3 — Tonnetz 격자 위 hibari의 pitch class



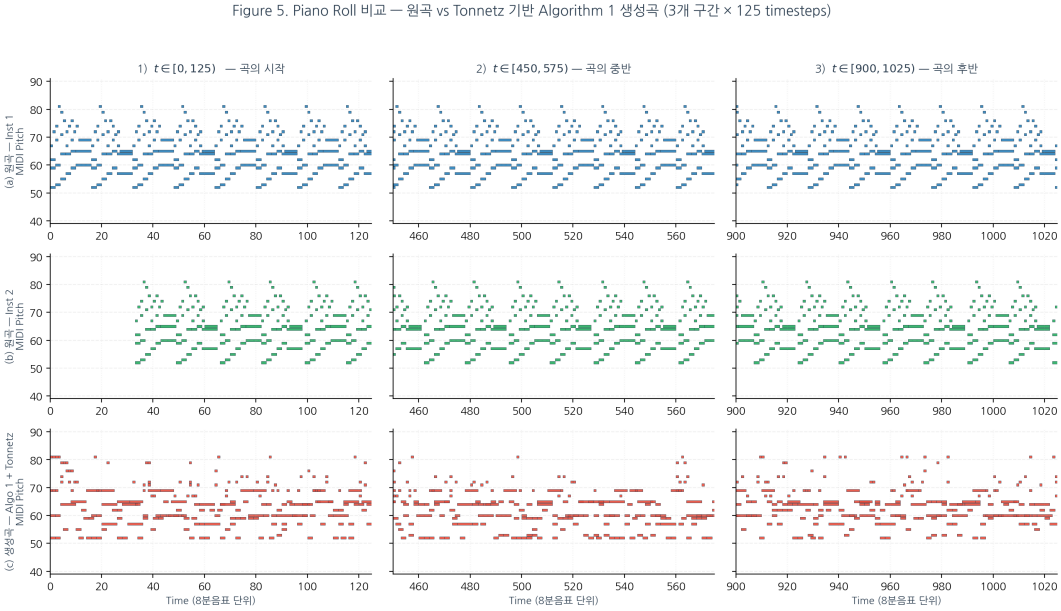
캡션. 12개의 pitch class가 배치된 Tonnetz 격자(가로: 완전 5도, 대각선: 장/단 3도) 위에서 hibari가 실제로 사용하는 7개 pitch class (C, D, E, F, G, A, B — C major scale)를 진한 파랑으로 강조하였다. 각 음의 크기는 곡 내 출현 빈도에 비례하며, 빨간 선은 사용된 pitch class 쌍 중 Tonnetz 그래프에서 직접 인접한 관계이다. 사용된 음들이 격자 상에서 연결 성분을 이루며 집중되어 있음을 확인할 수 있다.

Figure 4 — Persistence Barcode Diagram



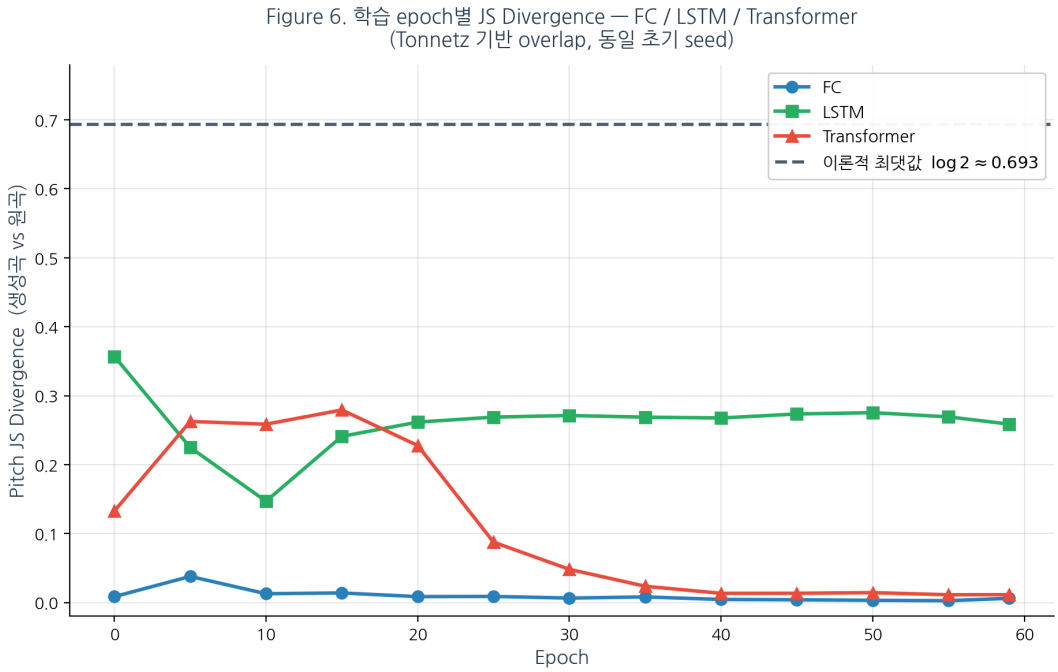
캡션. hibari의 H 1 cycle 전체 48 개에 대한 persistence barcode이다. 가로축은 rate parameter r_t, 각 막대는 한 cycle의 $[\mathrm{birth}, \mathrm{death}]$ 구간을 나타낸다. 48개 cycle은 lifespan 기준 내림차순으로 정렬되어 있어 위쪽에 오래 살아남은 cycle이 온다. 빨간색 막대는 탐색 구간 $r_t \in [0, 1.5]$ 내에서 소멸하지 않은 **영속 cycle** (총 7개, 원본 데이터에서 $\mathrm{death} \approx 10001$로 표기)이며, 오른쪽 끝의 ∞ 기호로 "탐색 범위 밖에서도 살아있음"을 표시한다. 나머지 41개 유한 cycle은 viridis colormap으로 lifespan에 비례하는 색으로 그려진다. 긴 막대는 rate 변화에 강건한, 즉 구조적으로 중요한 cycle을 의미한다.

Figure 5 — Piano Roll 비교 (3 구간)



캡션. 위에서부터 순서대로 (a) 원곡 악기 1, (b) 원곡 악기 2, (c) Algorithm 1 + Tonnetz (K=46, seed=42)로 생성된 곡이다. 전체 $T = 1088$ timesteps을 한 번에 보이면 해상도가 너무 낮아 개별 note가 식별되지 않으므로, 곡의 시작 $(t \in [0, 125))$, 중반 $(t \in [450, 575))$, 후반 $(t \in [900, 1025))$ 세 구간을 각 125 timesteps 씩 나란히 보여준다. 가로축은 시간(8분음표 단위), 세로축은 MIDI pitch이다. 각 구간에서 생성곡의 pitch 분포와 시간적 밀도가 원곡과 유사한 영역에서 활성화되고 있음을 시각적으로 확인할 수 있으며, 이는 3.1절의 JS divergence 정량 결과와 일관된다.

Figure 6 — 학습 Epoch별 JS Divergence 곡선



캡션. FC / LSTM / Transformer 세 모델을 동일한 데이터(Tonnetz overlap)와 동일 seed에서 60 epoch 학습시키면서, 매 5 epoch마다 생성곡을 만들어 원곡과의 pitch JS divergence를 측정한 결과이다. 진한 검정 파선은 이론적 최댓값 $\log 2 \approx 0.693$이다. FC 모델이 epoch 30 근처에서 $\mathrm{JS} \approx 0.003$에 도달하여 가장 빠르게 수렴하고, Transformer는 epoch 30 이후 같은 수준까지 따라잡는다. 반면 LSTM은 $\mathrm{JS} \approx 0.26$ 근처에서 plateau에 머물며 개선되지 않는다. 이는 3.4절 해석 8 ("hibari는 out of noise 앨범의 곡으로서 시간 문맥보다 음의 공간적 배치가 핵심")과 일관된 관찰이다.

재현 방법

```
cd tda_pipeline/docs/figures

#    figures (      )
python make_fig1_pipeline.py        #
python make_fig3_tonnetz_hibari.py  #
python make_fig4_barcode.py         # pickle/h1_rBD_*.pkl
python make_fig5_pianoroll.py       #       + cache    (~5s)
python make_fig6_js_curve.py        # fig6_curves.json        ,
                                    #            (~2-3 min)

# Interactive supplementary
python make_fig2_interactive.py     # Plotly HTML
```

모든 정적 figure는 200 dpi PNG로 저장되며, 한글 폰트는 `_fontsetup.py`에서 NanumGothic을 자동 등록한다. Interactive figure는 `plotly` 패키지가 필요하며, 생성된 HTML은 `include_plotlyjs='cdn'` 옵션으로

CDN에서 plotly.js를 로드하므로 오프라인에서는 별도 설정이 필요할 수 있다.